INDEX

Sl. No.	Programs	Page No.	Date	Remarks
1	Loading and Displaying Grayscale and Color Images; Color to Grayscale Conversion	4	15-06-2026	
2	Image Sampling and Quantization	5	18-06-2026	
3	Spatial and Intensity Resolution	7	23-06-2026	
4	Adjacency and Connectivity in Binary Images	10	30-06-2026	
5	RGB Component Separation and RGB to CMY and HSI Conversion	11	02-07-2026	
6	Modification of Hue, Saturation, and Intensity	15	06-07-2026	
7	Color Histograms for Individual Channels	16	09-07-2026	
8	Image Negative Transformation	17	13-07-2026	
9	Log Transformation and Contrast Enhancement	18	16-07-2026	
10	Power-Law (Gamma) Transformation	20	21-07-2026	
11	Piecewise Linear Transformations: Contrast Stretching and Thresholding	22	27-07-2026	
12	Histogram Equalization	23	31-07-2026	
13	Histogram Stretching	25	6-08-2026	
14	Histogram Matching (Specification)	27	10-08-2026	
15	Spatial Correlation and Convolution Using Custom Kernels	29	13-08-2026	
16	Smoothing Filters: Mean and Gaussian	31	17-08-2026	
17	Image Sharpening Using Laplacian Filter	32	18-08-2026	
18	Unsharp Masking and High-Boost Filtering	33	19-08-2026	
19	Edge Detection Using Sobel and Prewitt Operators	34	01-09-2026	
20	2D Fourier Transform and Inverse DFT Image Reconstruction	36	10-09-2026	
21	Low-Pass and High-Pass Filtering in the Frequency Domain	38	14-09-2026	
22	DFT Properties: Translation and Scaling	40	17-09-2026	
23	Morphological Operations: Dilation, Erosion, Opening, and Closing	42	22-09-2026	
24	Hit-or-Miss Transformation for Pattern Detection	44	25-10-2026	
25	Addition of Different Types of Noise and Noise Reduction Using Mean Filters	45	01-10-2026	

PROGRAM NO:1

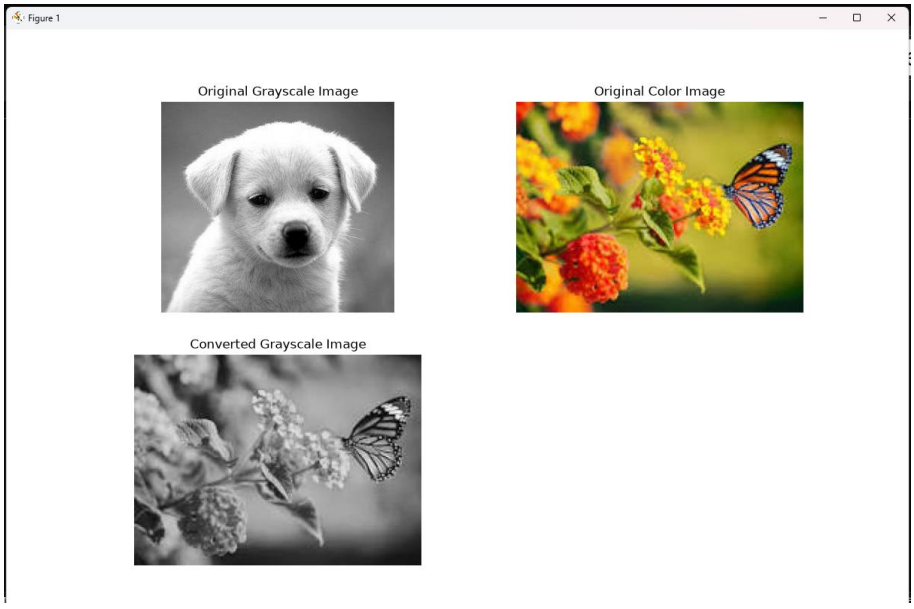
AIM:

Load a grayscale and color image and display, Convert a color image to grayscale.

PROGRAM

```
import cv2
# Load color image
color_image = cv2.imread("image.jpg")
# Load grayscale image
gray_image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
# Check images
if color_image is None or gray_image is None:
    print("Image not found. Check the file path.")
    exit()
# Display images
cv2.imshow("Color Image", color_image)
cv2.imshow("Grayscale Image", gray_image)
# Convert color image to grayscale
converted_gray = cv2.cvtColor(color_image, cv2.COLOR_BGR2GRAY)
cv2.imshow("Converted Grayscale", converted_gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:2

AIM:

Simulate image sampling and quantization by reducing resolution and gray levels.

PROGRAM

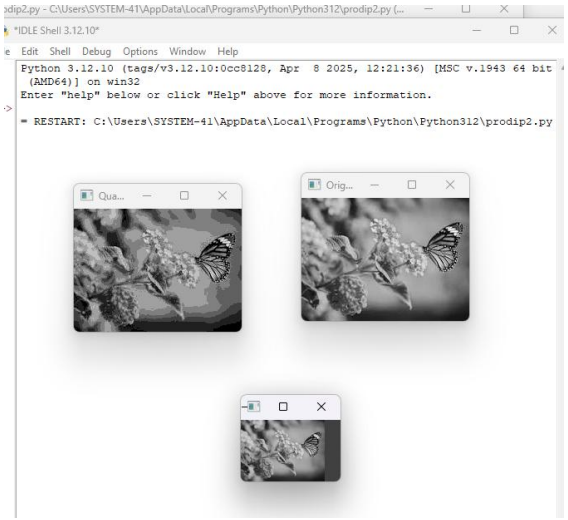
```
import cv2
import numpy as np
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
if img is None:
    print("Error: Image not found!")
    exit()
sampled = cv2.resize(img, (img.shape[1] // 2, img.shape[0] // 2))
levels = 8
step = 256 // levels
quantized = (img // step) * step
cv2.imshow("Original Image", img)
cv2.imshow("Sampled Image", sampled)
cv2.imshow("Quantized Image", quantized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT

Original Image
Resolution : 512 x 512
Gray Levels: 256

Sampled Image
Resolution : 256 x 256
Gray Levels: 256

Quantized Image
Resolution : 512 x 512
Gray Levels: 8



PROGRAM NO:3

AIM:

Demonstrate spatial and intensity resolution differences using image resizing and quantization.

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

small = cv2.resize(img, (100, 100))
large = cv2.resize(img, (400, 400))
img_4bit = (img // 16) * 16
img_2bit = (img // 64) * 64
plt.figure(figsize=(10,8))
plt.subplot(2,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(small, cmap='gray')
plt.title("100 x 100")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(large, cmap='gray')
plt.title("400 x 400")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(img, cmap='gray')
plt.title("8-bit (256 Levels)")
plt.axis("off")

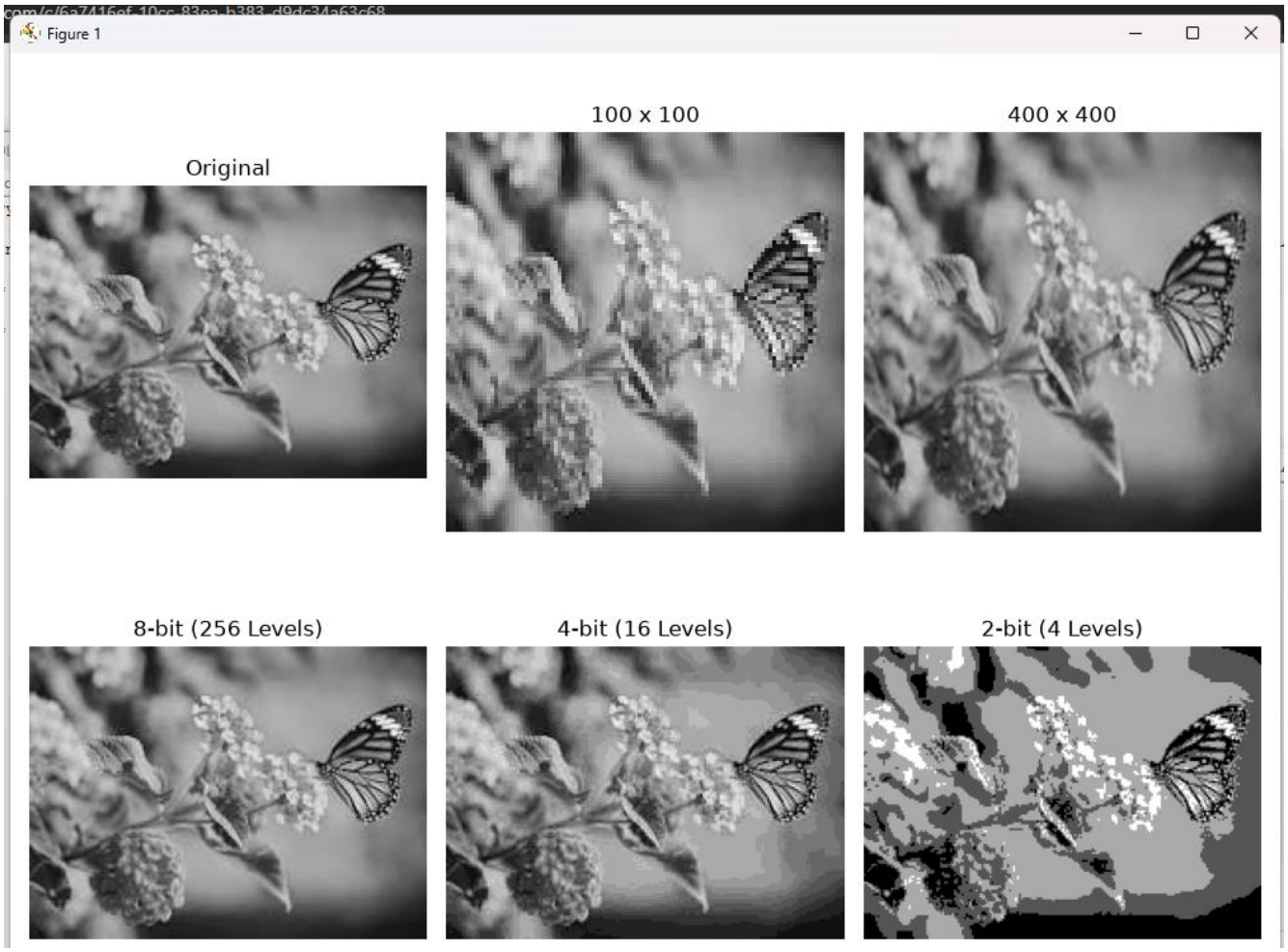
plt.subplot(2,3,5)
plt.imshow(img_4bit, cmap='gray')
plt.title("4-bit (16 Levels)")
plt.axis("off")

plt.subplot(2,3,6)
plt.imshow(img_2bit, cmap='gray')
plt.title("2-bit (4 Levels)")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:4

AIM:

Compute and visualize adjacency and connectivity in a binary image.

PROGRAM

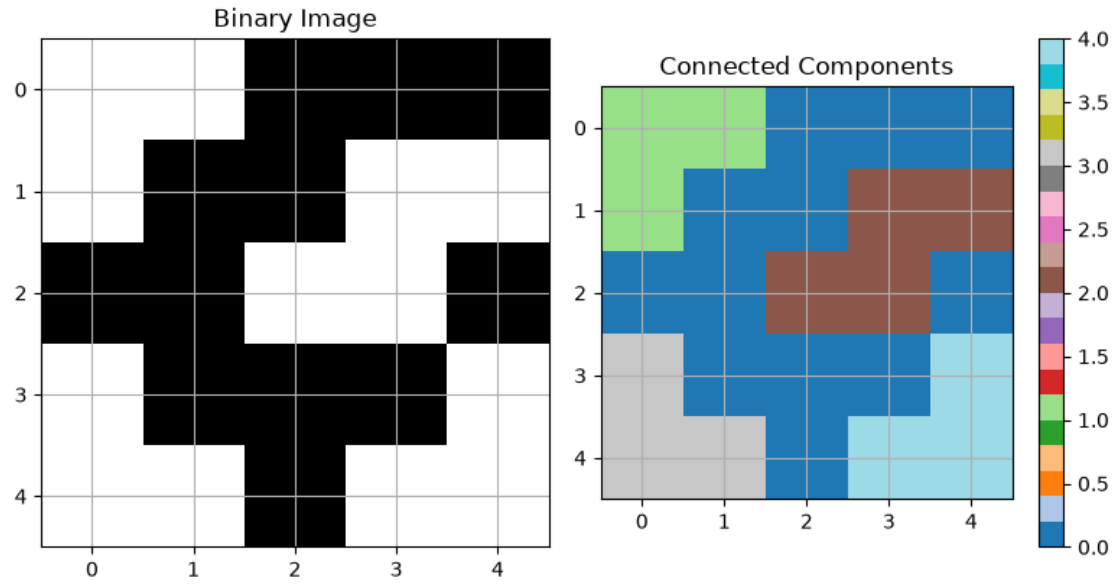
```
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
image = np.array([
    [1, 1, 0, 0, 0],
    [1, 0, 0, 1, 1],
    [0, 0, 1, 1, 0],
    [1, 0, 0, 0, 1],
    [1, 1, 0, 1, 1]
])
rows, cols = image.shape
directions = [(-1,0),(1,0),(0,-1),(0,1)]
visited = np.zeros_like(image, dtype=bool)
labels = np.zeros_like(image)
component = 0
for i in range(rows):
    for j in range(cols):
        if image[i][j] == 1 and not visited[i][j]:
            component += 1
            queue = deque([(i,j)])
            visited[i][j] = True
            while queue:
                x, y = queue.popleft()
                labels[x][y] = component
```

```
            for dx, dy in directions:
                nx, ny = x + dx, y + dy

                if (0 <= nx < rows and
                    0 <= ny < cols and
                    image[nx][ny] == 1 and
                    not visited[nx][ny]):
                    visited[nx][ny] = True
                    queue.append((nx, ny))

print("Binary Image:")
print(image)
print("\nConnected Components:")
print(labels)
plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray')
plt.title("Binary Image")
plt.grid(True)
plt.subplot(1,2,2)
plt.imshow(labels, cmap='tab20')
plt.title("Connected Components")
plt.colorbar()
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT




```
>>>
= RESTART: C:/Users/SYSTEM-41/AppData/Local/Programs/Python/Python312/prodip4.py
Binary Image:
[[1 1 0 0 0]
 [1 0 0 1 1]
 [0 0 1 1 0]
 [1 0 0 0 1]
 [1 1 0 1 1]]

Connected Components:
[[1 1 0 0 0]
 [1 0 0 2 2]
 [0 0 2 2 0]
 [3 0 0 0 4]
 [3 3 0 4 4]]
>>>
```

PROGRAM NO:5

AIM:

Read an image and separate it into RGB components. Convert RGB image to CMY and HSI color models.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("image.jpg")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
R = rgb[:, :, 0]
G = rgb[:, :, 1]
B = rgb[:, :, 2]
C = 255 - R
M = 255 - G
Y = 255 - B
r = R / 255.0
g = G / 255.0
b = B / 255.0
num = 0.5 * ((r - g) + (r - b))
den = np.sqrt((r - g) ** 2 + (r - b) * (g - b))
theta = np.arccos(np.clip(num / (den + 1e-6), -1, 1))
H = np.where(b <= g, theta, 2 * np.pi - theta)
H = H / (2 * np.pi)
S = 1 - (3 * np.minimum(np.minimum(r, g), b) / (r + g + b + 1e-6))
I = (r + g + b) / 3
plt.figure(figsize=(12,8))
plt.subplot(3,3,1)
plt.imshow(rgb)
```

```
plt.title("Original RGB")
plt.axis("off")

plt.subplot(3,3,2)
plt.imshow(R, cmap="Reds")
plt.title("Red")
plt.axis("off")

plt.subplot(3,3,3)
plt.imshow(G, cmap="Greens")
plt.title("Green")
plt.axis("off")

plt.subplot(3,3,4)
plt.imshow(B, cmap="Blues")
plt.title("Blue")
plt.axis("off")

plt.subplot(3,3,5)
plt.imshow(C, cmap="gray")
plt.title("Cyan (C)")
plt.axis("off")

plt.subplot(3,3,6)
plt.imshow(M, cmap="gray")
plt.title("Magenta (M)")
plt.axis("off")
plt.subplot(3,3,7)
plt.imshow(Y, cmap="gray")
```

```
plt.title("Yellow  (Y)")
plt.axis("off")

plt.subplot(3,3,8)
plt.imshow(H, cmap="hsv")
plt.title("Hue")
plt.axis("off")

plt.subplot(3,3,9)
plt.imshow(S, cmap="gray")
plt.title("Saturation")
plt.axis("off")
plt.tight_layout()
plt.show()

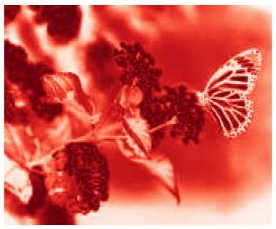

plt.imshow(I, cmap="gray")
plt.title("Intensity")
plt.axis("off")
plt.show()
```

OUTPUT

Original RGB



Red



Green



Blue



Cyan (C)



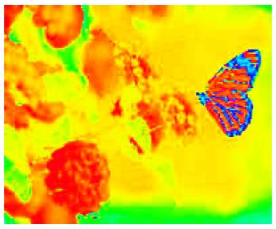
Magenta (M)



Yellow (Y)



Hue



Saturation



Intensity



PROGRAM NO:6

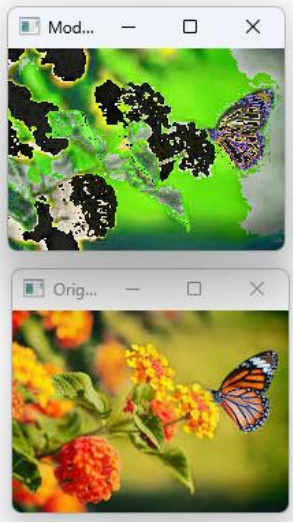
AIM:

Modify hue, saturation, and intensity and display the output.

PROGRAM

```
import cv2
import numpy as np
image = cv2.imread("image.jpg")
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)
h = (h + 20) % 180                # Change hue
s = np.clip(s + 50, 0, 255).astype(np.uint8)   # Increase saturation
v = np.clip(v + 40, 0, 255).astype(np.uint8)   # Increase intensity
modified_hsv = cv2.merge([h, s, v])
output = cv2.cvtColor(modified_hsv, cv2.COLOR_HSV2BGR)
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Modified Image", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:7

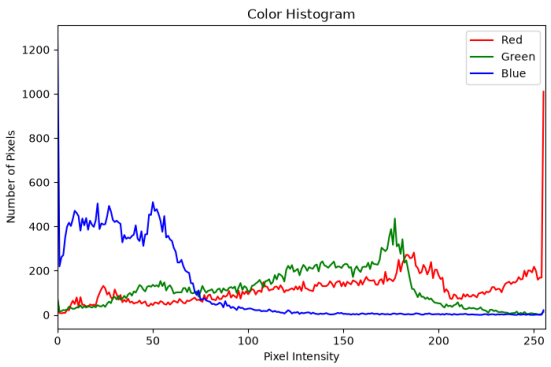
AIM:

Plot color histograms for each channel

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("image.jpg")
# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Colors and channel names
colors = ('r', 'g', 'b')
channels = ('Red', 'Green', 'Blue')
# Plot histogram for each channel
plt.figure(figsize=(8, 5))
for i, color in enumerate(colors):
    hist = cv2.calcHist([image_rgb], [i], None, [256], [0, 256])
    plt.plot(hist, color=color, label=channels[i])
plt.title("Color Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")
plt.xlim([0, 256])
plt.legend()
plt.show()
```

OUTPUT



PROGRAM NO:8

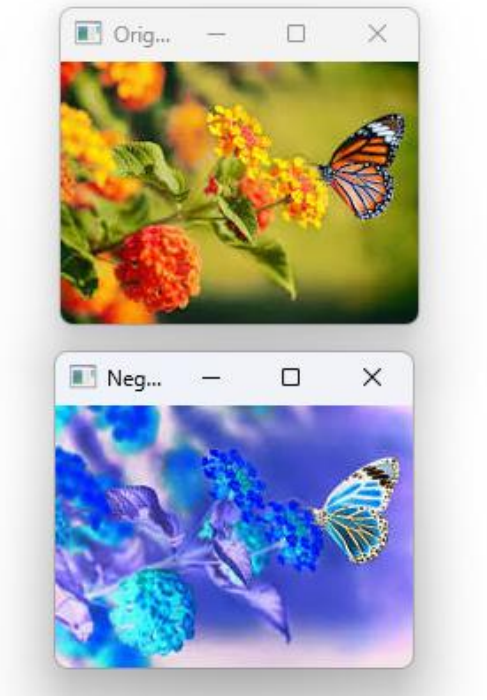
AIM:

Image negative transformation.

PROGRAM

```
import cv2
# Read the input image
image = cv2.imread("image.jpg")
# Create the negative image
negative = 255 - image
# Display the original and negative images
cv2.imshow("Original Image", image)
cv2.imshow("Negative Image", negative)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:9

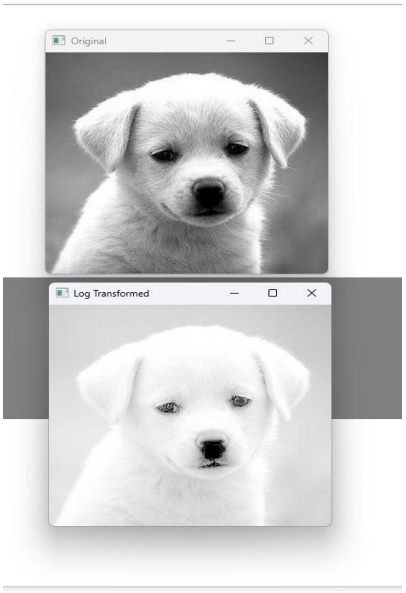
AIM:

Apply log transformation and analyse contrast enhancement

PROGRAM

```
import cv2
import numpy as np
image = cv2.imread("dark_image.jpg", 0)
if image is None:
    print("Error: Image not found.")
    exit()
log_image = np.log1p(image.astype(np.float32))
log_image = cv2.normalize(log_image, None, 0, 255, cv2.NORM_MINMAX)
log_image = np.uint8(log_image)
cv2.imshow("Original", image)
cv2.imshow("Log Transformed", log_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:10

AIM:

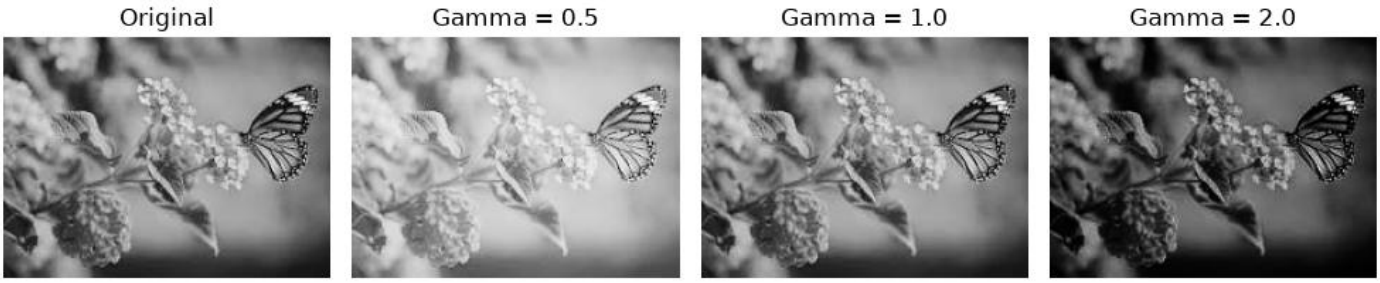
Power-law (gamma) transformation for different gamma values.

PRORAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
image = cv2.imread("image.jpg", 0)
# Check if image is loaded
if image is None:
    print("Error: Image not found.")
    exit()
# Normalize image to range [0,1]
image = image / 255.0
# Gamma values
gamma_values = [0.5, 1.0, 2.0]
# Display original image
plt.figure(figsize=(10, 4))
plt.subplot(1, 4, 1)
plt.imshow(image, cmap='gray')
plt.title("Original")
plt.axis("off")
# Apply gamma transformation
for i, gamma in enumerate(gamma_values):
    gamma_image = np.power(image, gamma)
    plt.subplot(1, 4, i + 2)
    plt.imshow(gamma_image, cmap='gray')
    plt.title(f"Gamma = {gamma}")
```

```
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:11

AIM:

Perform piecewise linear transformations (contrast stretching, thresholding).

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image in grayscale
image = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image not found!")
    exit()

r1 = 70
r2 = 180

# Output intensity range
s1 = 0
s2 = 255

# Create an empty output image
contrast = np.zeros_like(image)

# Piecewise linear transformation
for i in range(image.shape[0]):
    for j in range(image.shape[1]):
        r = image[i, j]
        if r <= r1:
            s = (s1 / r1) * r

        elif r <= r2:
            s = ((s2 - s1) / (r2 - r1)) * (r - r1) + s1

        else:
            s = ((255 - s2) / (255 - r2)) * (r - r2) + s2

        contrast[i, j] = np.clip(s, 0, 255)

threshold = 128
binary = np.zeros_like(image)
for i in range(image.shape[0]):
    for j in range(image.shape[1]):

        if image[i, j] >= threshold:
            binary[i, j] = 255
        else:
```

```
            binary[i, j] = 0

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(contrast, cmap="gray")
plt.title("Contrast Stretching")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(binary, cmap="gray")
plt.title("Thresholding")
plt.axis("off")
plt.tight_layout()
plt.show()
# Save output images
cv2.imwrite("contrast_output.jpg", contrast)
cv2.imwrite("threshold_output.jpg", binary)
print("Contrast stretching completed.")
print("Thresholding completed.")
print("Output images saved successfully.")
```

OUTPUT



PROGRAM NO:12

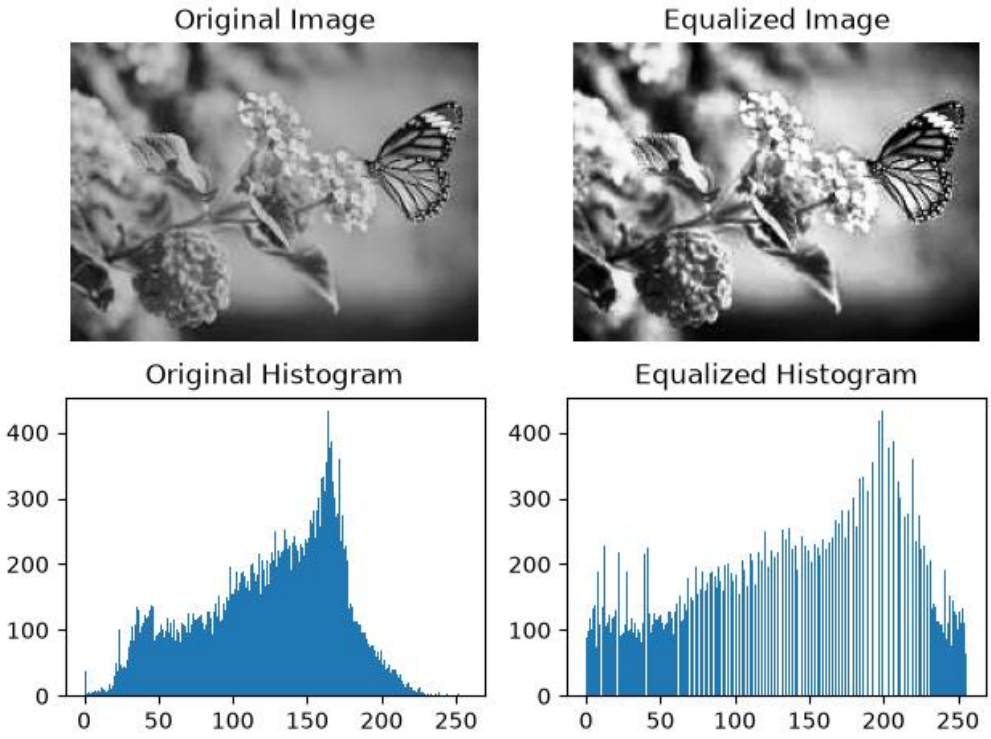
AIM:

Perform histogram equalization and compare results.

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
# Read image
img = cv2.imread("image.jpg", 0)
# Histogram equalization
equalized = cv2.equalizeHist(img)
# Display images
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(equalized, cmap="gray")
plt.title("Equalized Image")
plt.axis("off")
# Display histograms
plt.subplot(2, 2, 3)
plt.hist(img.ravel(), 256, [0, 256])
plt.title("Original Histogram")
plt.subplot(2, 2, 4)
plt.hist(equalized.ravel(), 256, [0, 256])
plt.title("Equalized Histogram")
plt.tight_layout()
plt.show()
```

OUTPUT


Original Image
Equalized Image
Original Histogram
Equalized Histogram
400
300
200
100
0
0
50
100
150
200
250

PROGRAM NO:13

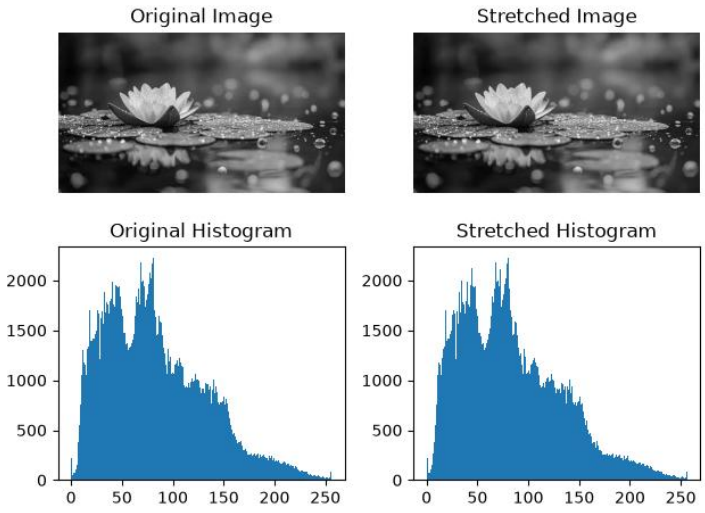
AIM:

Implement histogram stretching.

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("input.jpg", 0)
# Histogram stretching
min_val = img.min()
max_val = img.max()
stretched = ((img - min_val) / (max_val - min_val) * 255).astype("uint8")
# Display images
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(stretched, cmap="gray")
plt.title("Stretched Image")
plt.axis("off")
# Display histograms
plt.subplot(2, 2, 3)
plt.hist(img.ravel(), 256, [0, 256])
plt.title("Original Histogram")
plt.subplot(2, 2, 4)
plt.hist(stretched.ravel(), 256, [0, 256])
plt.title("Stretched Histogram")
plt.tight_layout()
plt.show()
```

OUTPUT


Original Image
Stretched Image
Original Histogram
Stretched Histogram
2000
1500
1000
500
0
0
50
100
150
200
250

PROGRAM NO:14

AIM:

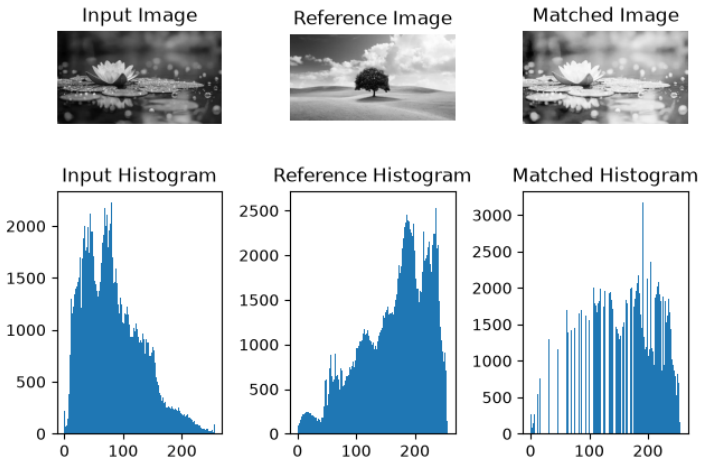
Apply histogram matching (specification) with a reference image.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read images in grayscale
input_img = cv2.imread("input.jpg", 0)
reference_img = cv2.imread("reference.jpg", 0)
# Calculate histograms
hist_input = cv2.calcHist([input_img], [0], None, [256], [0, 256])
hist_ref = cv2.calcHist([reference_img], [0], None, [256], [0, 256])
# Calculate CDF
cdf_input = hist_input.cumsum()
cdf_ref = hist_ref.cumsum()
# Normalize CDF
cdf_input = cdf_input / cdf_input[-1]
cdf_ref = cdf_ref / cdf_ref[-1]
# Create mapping
mapping = np.zeros(256, dtype=np.uint8)
for i in range(256):
    mapping[i] = np.argmin(abs(cdf_ref - cdf_input[i]))
matched_img = mapping[input_img]
# Display images
plt.subplot(2, 3, 1)
plt.imshow(input_img, cmap="gray")
plt.title("Input Image")
plt.axis("off")
```

```
plt.subplot(2, 3, 2)
plt.imshow(reference_img, cmap="gray")
plt.title("Reference Image")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(matched_img, cmap="gray")
plt.title("Matched Image")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.hist(input_img.ravel(), 256, [0, 256])
plt.title("Input Histogram")
plt.subplot(2, 3, 5)
plt.hist(reference_img.ravel(), 256, [0, 256])
plt.title("Reference Histogram")
plt.subplot(2, 3, 6)
plt.hist(matched_img.ravel(), 256, [0, 256])
plt.title("Matched Histogram")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:15

AIM:

Perform spatial correlation and convolution using custom kernels.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("image.jpg", 0)
# Custom 3x3 kernel
kernel = np.array([
    [1, 1, 1],
    [1, 1, 1],
    [1, 1, 1]
], dtype=np.float32) / 9
# Spatial correlation
correlation = cv2.filter2D(img, -1, kernel)
# Spatial convolution
convolution = cv2.filter2D(img, -1, np.flip(kernel))
# Display results
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(correlation, cmap="gray")
plt.title("Correlation")
plt.axis("off")
plt.subplot(1, 3, 3)
```

```
plt.imshow(convolution, cmap="gray")
plt.title("Convolution")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:16

AIM:

Apply smoothing filters (mean, Gaussian).

PROGRAM

```
import cv2
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("image.jpg", 0)
# Apply Mean filter
mean = cv2.blur(img, (5, 5))
# Apply Gaussian filter
gaussian = cv2.GaussianBlur(img, (5, 5), 0)
# Display images
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off"
plt.subplot(1, 3, 2)
plt.imshow(mean, cmap="gray")
plt.title("Mean Filter")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:17

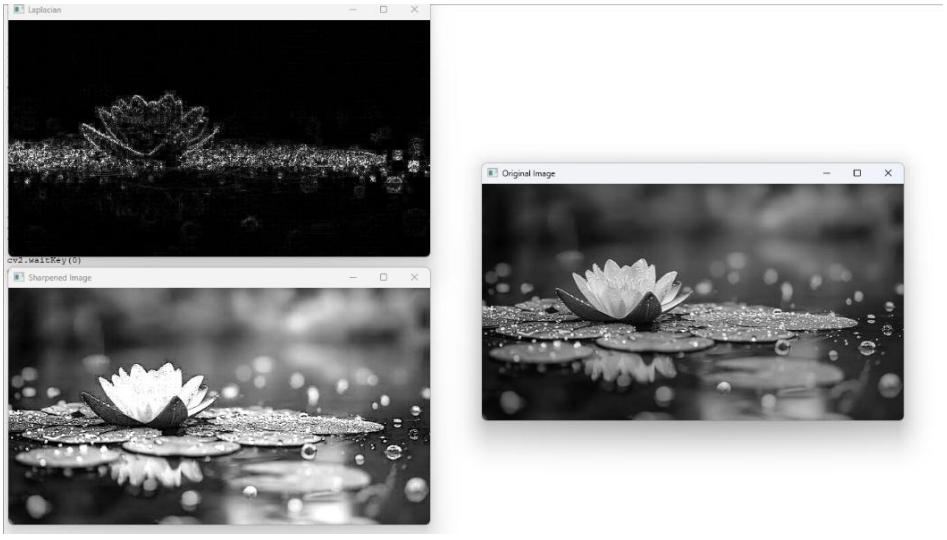
AIM:

Sharpening filters using Laplacian

PROGRAM

```
import cv2
import numpy as np
# Read the image
image = cv2.imread("input.jpg")
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Laplacian filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
# Convert Laplacian result to 8-bit
laplacian = cv2.convertScaleAbs(laplacian)
# Sharpen the image
sharpened = cv2.addWeighted(gray, 1.5, laplacian, -0.5, 0)
# Display images
cv2.imshow("Original Image", gray)
cv2.imshow("Laplacian", laplacian)
cv2.imshow("Sharpened Image", sharpened)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:18

AIM:

Perform unsharp masking and high-boost filtering.

PROGRAM

```
import cv2
# Read the image
img = cv2.imread("input.jpg")
# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Blur the image
blur = cv2.GaussianBlur(gray, (5, 5), 0)
mask = cv2.subtract(gray, blur)
unsharp = cv2.add(gray, mask)
A = 2.0
high_boost = cv2.addWeighted(gray, A, blur, -1, 0)
cv2.imshow("Original Image", gray)
cv2.imshow("Unsharp Masking", unsharp)
cv2.imshow("High-Boost Filtering", high_boost)
cv2.imwrite("unsharp.jpg", unsharp)
cv2.imwrite("high_boost.jpg", high_boost)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:19

AIM:

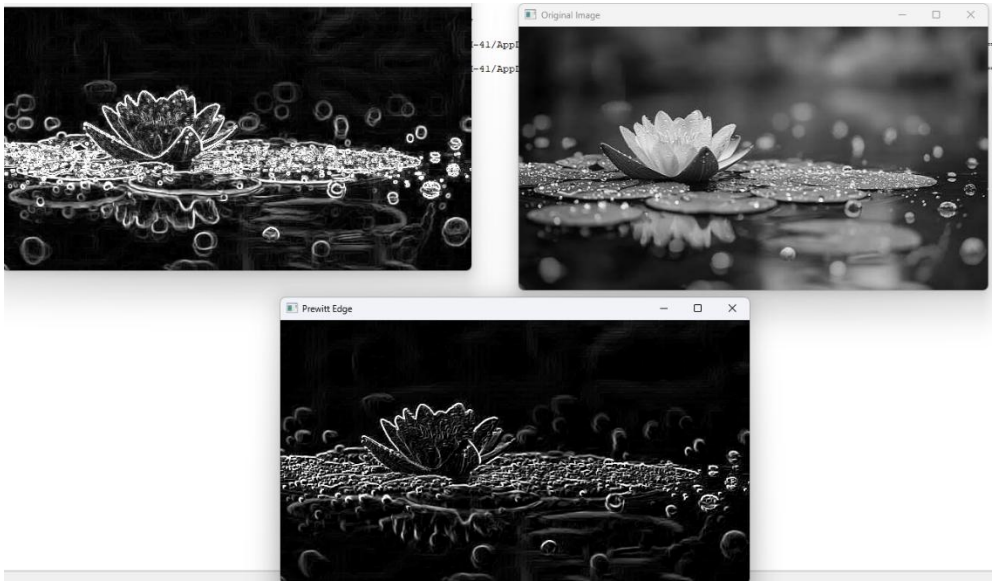
Apply gradient operators (Sobel, Prewitt) for edge detection.

PROGRAM

```
import cv2
import numpy as np
# Read image
img = cv2.imread("input.jpg", 0)
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = np.uint8(np.clip(sobel, 0, 255))
prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])
prewitt_y = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])
px = cv2.filter2D(img, -1, prewitt_x)
py = cv2.filter2D(img, -1, prewitt_y)
prewitt = cv2.magnitude(
    px.astype(np.float32),
    py.astype(np.float32)
)
prewitt = np.uint8(np.clip(prewitt, 0, 255))
```

```
# Display results
cv2.imshow("Original Image", img)
cv2.imshow("Sobel Edge", sobel)
cv2.imshow("Prewitt Edge", prewitt)
# Save results
cv2.imwrite("sobel.jpg", sobel)
cv2.imwrite("prewitt.jpg", prewitt)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



PROGRAM NO:20

AIM:

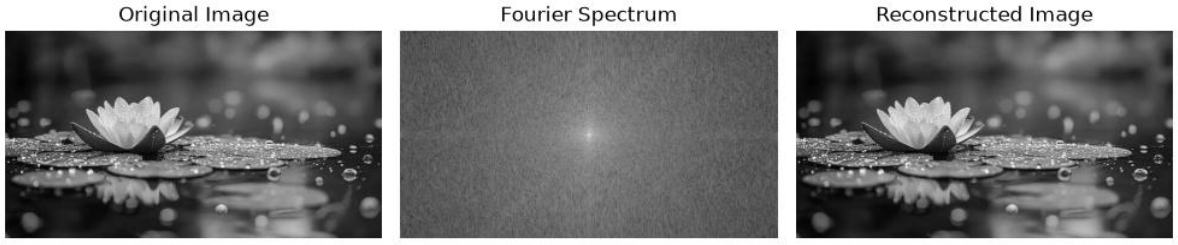
Compute and visualize the 2D Fourier Transform of an image. Implement inverse DFT and reconstruct the image.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("input.jpg", 0)
dft = np.fft.fft2(img)
# Shift low frequencies to the center
dft_shift = np.fft.fftshift(dft)
# Calculate magnitude spectrum
magnitude = np.abs(dft_shift)
# Use log scale for visualization
spectrum = np.log(1 + magnitude)
dft_inverse_shift = np.fft.ifftshift(dft_shift)
# Perform inverse DFT
reconstructed = np.fft.ifft2(dft_inverse_shift)
# Convert complex result to image
reconstructed = np.abs(reconstructed)
reconstructed = np.uint8(np.clip(reconstructed, 0, 255))
plt.figure(figsize=(10, 5))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(spectrum, cmap="gray")
plt.title("Fourier Spectrum")
plt.axis("off")
plt.subplot(1, 3, 3)
```

```
plt.imshow(reconstructed,  cmap="gray")
plt.title("Reconstructed  Image")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:21

AIM:

Apply low-pass and high-pass filters in frequency domain.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("input.jpg", 0)
# Fourier Transform
dft = np.fft.fft2(img)
dft_shift = np.fft.fftshift(dft)
# Get image size
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
# Create a mask
mask = np.zeros((rows, cols), np.uint8)
# Filter size
r = 30
mask[crow-r:crow+r, ccol-r:ccol+r] = 1
# Apply Low-Pass Filter
low_pass = dft_shift * mask
# Inverse Fourier Transform
low_pass = np.fft.ifftshift(low_pass)
low_pass_img = np.fft.ifft2(low_pass)
low_pass_img = np.abs(low_pass_img)
high_mask = 1 - mask
# Apply High-Pass Filter
high_pass = dft_shift * high_mask
```

```
# Inverse Fourier Transform
high_pass = np.fft.ifftshift(high_pass)
high_pass_img = np.fft.ifft2(high_pass)
high_pass_img = np.abs(high_pass_img)
plt.figure(figsize=(10, 6))
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(low_pass_img, cmap="gray")
plt.title("Low-Pass Filter")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(high_pass_img, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:22

AIM:

Demonstrate properties of DFT (translation, scaling).

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("input.jpg", 0)
dft = np.fft.fft2(img)
spectrum = np.log(1 + np.abs(np.fft.fftshift(dft)))
shifted_img = np.roll(img, (30, 50), axis=(0, 1))
shifted_dft = np.fft.fft2(shifted_img)
shifted_spectrum = np.log(
   1 + np.abs(np.fft.fftshift(shifted_dft))
)
small_img = cv2.resize(img, None, fx=0.5, fy=0.5)
# DFT of scaled image
small_dft = np.fft.fft2(small_img)
small_spectrum = np.log(
   1 + np.abs(np.fft.fftshift(small_dft))
)
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(shifted_img, cmap="gray")
plt.title("Translated Image")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(small_img, cmap="gray")
plt.title("Scaled Image")
plt.axis("off")

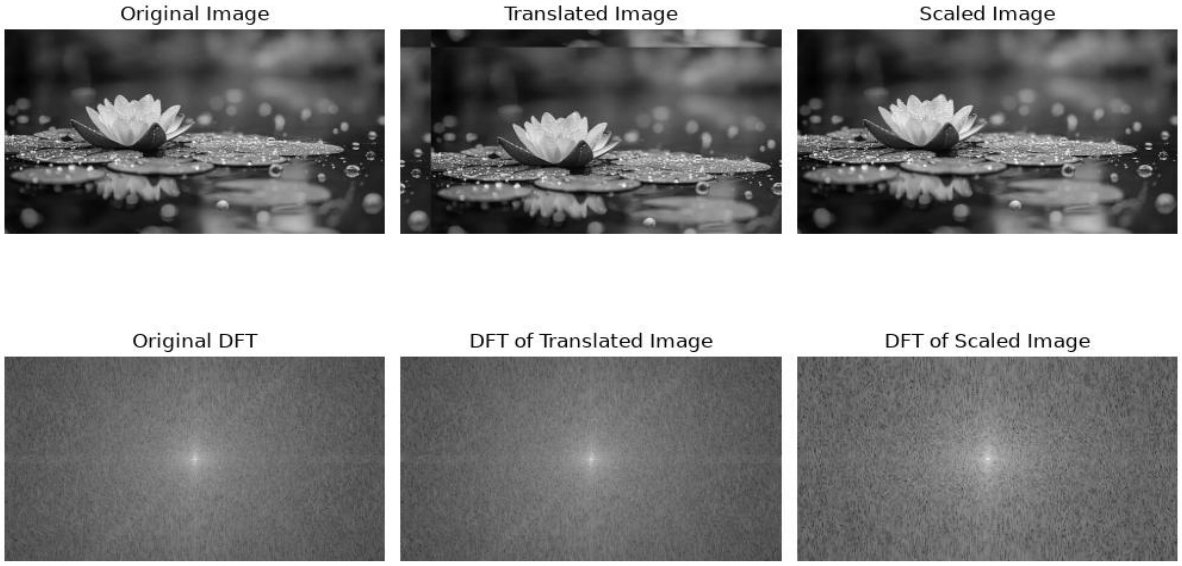
plt.subplot(2, 3, 4)
plt.imshow(spectrum, cmap="gray")
plt.title("Original DFT")
plt.axis("off")
```

```
plt.subplot(2, 3, 5)
plt.imshow(shifted_spectrum, cmap="gray")
plt.title("DFT of Translated Image")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(small_spectrum, cmap="gray")
plt.title("DFT of Scaled Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:23

AIM:

Perform basic morphological operations: dilation and erosion. Use different structuring elements and analyze effects,Opening and closing operations.

PROGRAM

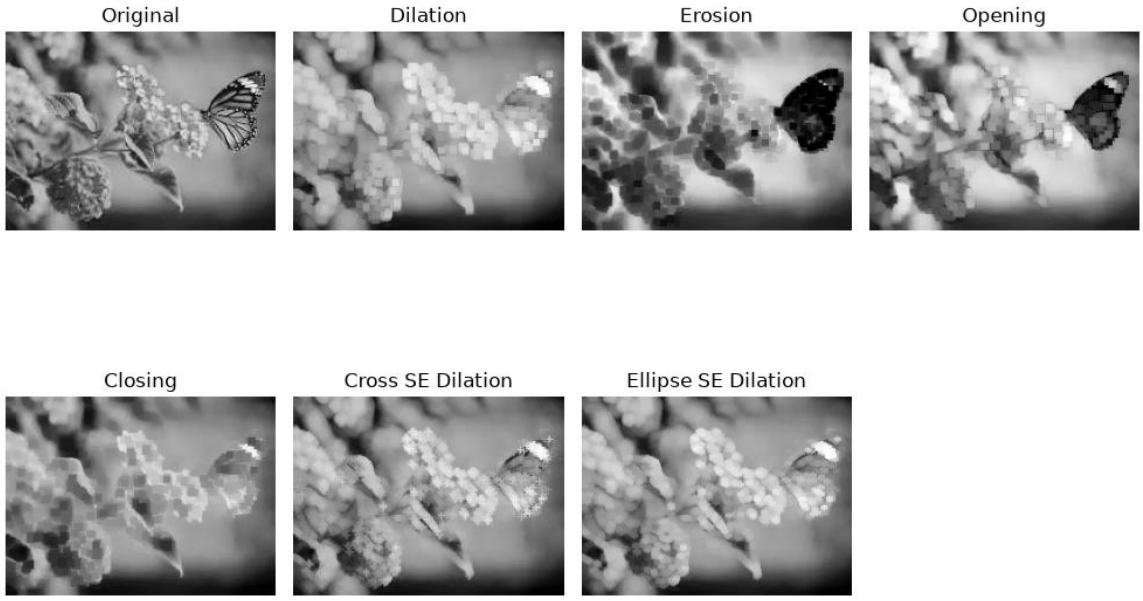
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image in grayscale
img = cv2.imread("image.jpg", 0)
# Different structuring elements
square = np.ones((5, 5), np.uint8)
cross = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5))
ellipse = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
# Morphological operations
dilation = cv2.dilate(img, square, iterations=1)
erosion = cv2.erode(img, square, iterations=1)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, square)
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, square)
# Display results
titles = [
    "Original", "Dilation", "Erosion",
    "Opening", "Closing", "Cross SE Dilation",
    "Ellipse SE Dilation"
]

images = [
    img,
    dilation,
    erosion,
    opening,
    closing,
    cv2.dilate(img, cross, iterations=1),
```

```
    cv2.dilate(img, ellipse, iterations=1)
]
plt.figure(figsize=(10, 7))
for i in range(len(images)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT



PROGRAM NO:24

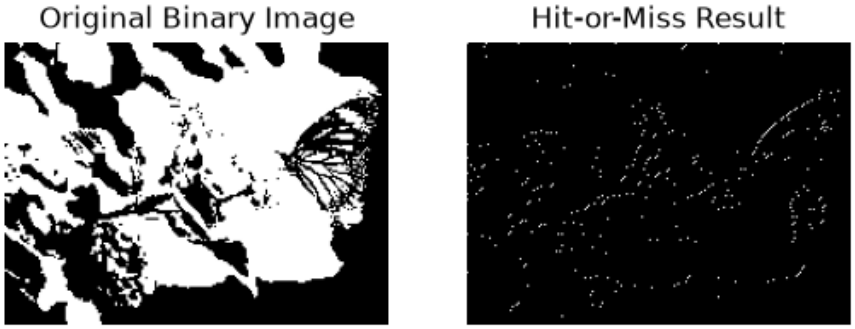
AIM:

Hit-or-miss transformation for pattern detection.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read the binary image
img = cv2.imread("image.jpg", 0)
# Convert image to binary
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
# Define structuring elements
# 1 = foreground, -1 = background, 0 = don't care
kernel = np.array([
    [ 1,  1,  0],
    [ 1, -1,  0],
    [ 0,  0,  0]
], dtype=np.int8)
    result = cv2.morphologyEx(binary, cv2.MORPH_HITMISS, kernel)
plt.subplot(1, 2, 1)
plt.imshow(binary, cmap="gray")
plt.title("Original Binary Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(result, cmap="gray")
plt.title("Hit-or-Miss Result")
plt.axis("off")
plt.show()
```

OUTPUT



PROGRAM NO:25

AIM:

Add different types of noise: Gaussian, Salt & Pepper (Impulse), Rayleigh / Gamma / Exponential, Apply mean filters and analyze results.

PROGRAM

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image
img = cv2.imread("image.jpg", 0)
# Convert image to float
image = img.astype(np.float32) / 255.0
# 1. Gaussian Noise
gaussian = image + np.random.normal(0, 0.1, image.shape)
gaussian = np.clip(gaussian, 0, 1)
# 2. Salt and Pepper Noise
sp = image.copy()
prob = 0.05
# Salt noise
salt = np.random.random(image.shape) < prob / 2
sp[salt] = 1
# Pepper noise
pepper = np.random.random(image.shape) < prob / 2
sp[pepper] = 0

# 3. Rayleigh Noise
rayleigh = image + np.random.rayleigh(0.1, image.shape)
rayleigh = np.clip(rayleigh, 0, 1)

# 4. Gamma Noise
gamma = image + np.random.gamma(1, 0.1, image.shape)
gamma = np.clip(gamma, 0, 1)

# 5. Exponential Noise
exponential = image + np.random.exponential(0.1, image.shape)
exponential = np.clip(exponential, 0, 1)

# Convert noisy images to uint8
gaussian = (gaussian * 255).astype(np.uint8)
sp = (sp * 255).astype(np.uint8)
rayleigh = (rayleigh * 255).astype(np.uint8)
gamma = (gamma * 255).astype(np.uint8)
```

```
exponential = (exponential * 255).astype(np.uint8)

# Apply mean filter
gaussian_filter = cv2.blur(gaussian, (5, 5))
sp_filter = cv2.blur(sp, (5, 5))
rayleigh_filter = cv2.blur(rayleigh, (5, 5))
gamma_filter = cv2.blur(gamma, (5, 5))
exponential_filter = cv2.blur(exponential, (5, 5))

# Display results
images = [
    img, gaussian, gaussian_filter,
    sp, sp_filter,
    rayleigh, rayleigh_filter,
    gamma, gamma_filter,
    exponential, exponential_filter
]

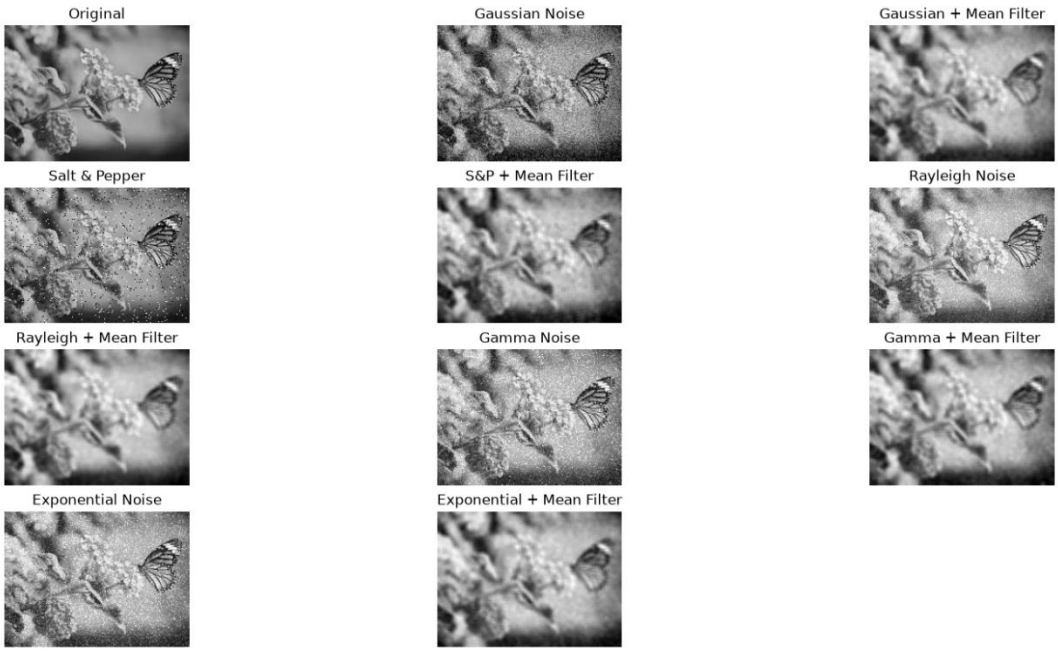
titles = [
    "Original",
    "Gaussian Noise", "Gaussian + Mean Filter",
    "Salt & Pepper", "S&P + Mean Filter",
    "Rayleigh Noise", "Rayleigh + Mean Filter",
    "Gamma Noise", "Gamma + Mean Filter",
    "Exponential Noise", "Exponential + Mean Filter"
]

plt.figure(figsize=(12, 9))

for i in range(len(images)):
    plt.subplot(4, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT


Original
Gaussian Noise
Gaussian + Mean Filter
Salt & Pepper
S&P + Mean Filter
Rayleigh Noise
Rayleigh + Mean Filter
Gamma Noise
Gamma + Mean Filter
Exponential Noise
Exponential + Mean Filter